

MERN interview handbook

A study and practice reference for product companies and startups

This handbook contains ten revision guides, 100 answered questions classified by learning difficulty, 24 debugging scenarios, and 16 machine coding approaches. Use the repository for runnable projects, mind maps, company playbooks, and the progress tracker.

How to study

Read one idea, close the guide, explain it aloud, implement an example, and demonstrate a boundary case. Score recall from 0 to 3: cannot explain, definition only, correct with example, or correct with tradeoff and failure case. Revisit weak answers after 1, 3, 7, and 14 days.

Scope and sources

Questions and scenarios are original practice, not leaked company questions. No material guarantees interview selection. External resources retain their licenses; links are included without reproducing external ebooks. The code examples are learning references and may omit a surrounding application setup explicitly provided in the repository.

Repository: [NEC MERN learning resources](#)

Contents

Part 1 Structured revision notes

Part 2 Difficulty based questions and answers

Part 3 Scenario diagnosis

Part 4 Machine coding approaches

Part 5 External learning references

Part 1 Structured revision notes

Web foundations and semantic HTML

Follow a request

A browser resolves a hostname, establishes a connection, and sends an HTTP request. HTTPS adds authenticated encryption. The server sends a status, headers, and a body; the browser parses HTML and discovers other resources. DNS, transport, TLS, and application errors have different diagnostic evidence.

Use the Network panel to distinguish a failed request from a response your code mishandled. Check status and content type before parsing JSON. A 200 response containing an HTML login page can still fail JSON parsing.

Build meaningful documents

Use one clear page title, ordered headings, landmarks, and native links and buttons. A link navigates to a location; a button performs an action. Add alt text that serves the image purpose, and use empty alt for decorative images.

A label is persistent instruction; a placeholder disappears when typing. Connect validation errors to controls and retain the user's input. Browser validation does not replace server checks.

Progressive enhancement

Start with useful content and navigation. Add richer interactions without making the core information impossible to reach if JavaScript fails. For an app requiring JavaScript, provide a clear loading and failure experience.

Test keyboard access, zoom, narrow widths, and slow connections. Accessibility is part of correctness, not a last-minute styling task.

Worked example

```
<form>
  <label for="email">Email address</label>
  <input id="email" name="email" type="email" autocomplete="email" required>
  <button type="submit">Join newsletter</button>
</form>
```

Demonstrate understanding

Build a registration form. Demonstrate labels, error associations, keyboard submission, and preserved input after failure.

Primary reference: https://developer.mozilla.org/en-US/docs/Learn_web_development

CSS layout and responsive interfaces

Reason about the layout

Use the box model and normal flow before adding positioning. A fixed width can overflow when padding is added under content-box sizing. Apply border-box deliberately, constrain images, and let long text wrap.

Choose flex for distribution along an axis and grid for track alignment. `min-width:0` on flex or grid children and `minmax(0,1fr)` on grid tracks often matter when content would otherwise force overflow.

Debug styles with evidence

Inspect computed styles and the winning declaration. Stacking contexts constrain z-index comparisons. Increasing a number does not allow a descendant to escape its ancestor context.

Avoid unexplained overrides. Use a small spacing and typography system, visible focus outlines, and a layout that survives 200 percent zoom. Respect reduced-motion preferences.

Design responsive behavior

Start with the smallest useful layout and add columns when content has room. Breakpoints should follow the content requirements. Use fluid widths and sensible max-widths rather than targeting a specific device brand.

Reserve media space to reduce shifts. Test unusually long labels and empty states as well as ideal sample data.

Worked example

```
* { box-sizing: border-box; }
.cards { display: grid; gap: 1rem; grid-template-columns: repeat(auto-fit,minmax(min(100%,16rem),1fr)); }
img { max-width: 100%; height: auto; }
:focus-visible { outline: 3px solid #174bc9; outline-offset: 3px; }
```

Demonstrate understanding

Build a responsive card grid and toolbar. Verify at 320px, 768px, and 200 percent zoom with long content.

Primary reference: <https://web.dev/learn/css>

JavaScript values closures and asynchronous work

Model values and identities

Objects are compared by identity; two objects with equal-looking properties are distinct. `const` prevents binding reassignment while nested mutation remains possible. Copy only the parts you change and define how missing, null, zero, and empty string differ in your domain.

Use explicit conversions for user input. `Number("")` is zero, which is often not an intended amount. Validate the original text and reject nonfinite values.

Use lexical scope

Closures retain access to their defining environment. A callback can retain large data or capture an earlier value. Normal functions get this from how they are called; arrows capture it from their surroundings.

Practice tracing small examples before running them. Explain the invocation, environment, and value rather than memorizing console output.

Control asynchronous operations

Promises describe eventual completion. Awaiting an operation suspends that async function, not the entire runtime. `Promise.all` preserves input ordering but does not cancel peers after failure.

Debounce user input, cancel obsolete requests, and bound parallel work. Timeouts and retry policies need an explicit contract. Treat a timeout of a write as an uncertain outcome until the server confirms it.

Worked example

```
console.log("A");
setTimeout(() => console.log("D"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("B");
// In a normal browser script: A B C D.
```

Demonstrate understanding

Implement debounce, LRU, and a promise pool using the contracts and tests in the JavaScript project.

Primary reference: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>

React state effects and resilient UI

Choose one state owner

Represent each fact in one authoritative place. Derive filtered results and totals during render. Keep drafts local to forms and remote data in a request-aware model. Lift shared state only as far as required.

Stable keys express identity. A changed key can intentionally reset state, while an array index can preserve the wrong row state after reordering.

Make transitions explicit

Functional state updates express a transition from the previous state. A reducer is useful when several values move together. Pure render logic lets React call components without performing hidden side effects.

Do not trigger a payment or mutation simply because a component rendered. User intent belongs in handlers; effects synchronize external systems and clean up subscriptions.

Handle remote state

Show loading, empty, success, and error states separately. Prevent old requests from replacing newer results using cleanup and request identity. Handle HTTP errors explicitly because fetch resolves on many error status codes.

Optimistic writes need rollback or refetch and a conflict policy. Client abort does not prove the server canceled a write. Use versions or mutation ids for correctness.

Worked example

```
useEffect(() => {
  const controller = new AbortController();
  let current = true;
  setState({status: "loading"});
  fetch(`/api/items?q=${encodeURIComponent(query)}`, {signal: controller.signal})
    .then(r => { if (!r.ok) throw new Error("Request failed"); return r.json(); })
    .then(data => { if (current) setState({status: "success", data}); })
    .catch(error => { if (current && error.name !== "AbortError") setState({status: "error",
      error}); });
  return () => { current = false; controller.abort(); };
}, [query]);
```

Demonstrate understanding

Explain each state transition in the MERN workspace and demonstrate an API failure without losing the form draft.

Primary reference: <https://react.dev/learn>

Node and Express request architecture

Separate transport and business rules

The route handles HTTP concerns. A service enforces domain rules, and a repository provides persistence. This separation makes route behavior testable without implying that a fake repository proves database behavior.

Choose one module system per package. State runtime requirements and commit dependency locks. Keep server secrets in process configuration and out of client bundles.

Budget resources

Nonblocking I/O allows useful work while other operations wait. Large synchronous computations still delay unrelated requests. Set request size limits, query bounds, timeouts, and rate limits.

Streams require backpressure and error handling. For CPU work, consider worker threads or background workers after measuring the actual bottleneck.

Control failure paths

Use one error handler that returns safe consistent errors. Express major versions differ for rejected async handlers. Log unexpected errors with correlation ids, while avoiding secrets.

Stop accepting requests during shutdown, finish in-flight work within a deadline, and close connections. Health endpoints should distinguish process health from dependency readiness.

Worked example

```
app.use(express.json({limit: "16kb"}));
app.get("/api/health", (req, res) => res.json({ok: true}));
// Express 5 forwards a returned rejected promise to error middleware.
app.use((error, req, res, next) => {
  if (res.headersSent) return next(error);
  res.status(500).json({error: "INTERNAL_ERROR"});
});
```

Demonstrate understanding

Add request ids and a structured logger to the reference API. Prove passwords and cookies are redacted.

Primary reference: <https://nodejs.org/en/learn/getting-started/introduction-to-nodejs>

MongoDB modeling indexes and concurrency

Model access and invariants

List the reads, writes, ownership rules, and growth bounds before choosing embedded or referenced data. A document is an atomic write boundary. Keep collections and arrays bounded where the domain allows it.

Mongoose helps express schemas, but a schema alone does not create the right query indexes or authorization rules. Write migrations for data and index changes.

Measure query work

Use explain on representative data. Compare examined keys and documents with returned rows and inspect whether sort work is supported by an index. Indexes improve some reads at a storage and write cost.

For a user-owned timeline, equality on owner followed by an ordered timestamp and id can support a useful compound index. Large skip values deserve a cursor design.

Protect concurrent writes

A read followed by an unconditional update can lose another edit. Include expected version in the update filter and increment version in the same operation. Use conditional stock updates for a single inventory document.

Transactions coordinate multi-document invariants but require an appropriate deployment. External side effects still need idempotency and compensation. The JavaScript event loop cannot enforce consistency across multiple processes.

Worked example

```
const result = await tasks.updateOne(
  {_id: id, owner: userId, version: expectedVersion},
  {$set: {status: "done"}, $inc: {version: 1}}
);
if (result.matchedCount === 0) {
  // Resolve an owned missing item versus an owned version conflict safely.
}
```

Demonstrate understanding

Run two updates with the same version against MongoDB. Exactly one must match, and the persisted version increases once.

Primary reference: <https://www.mongodb.com/docs/manual/core/write-operations-atomicity/>

Sessions API security and ownership

Establish trusted identity

Use a password-specific hash and unique salt. With server sessions, store a digest of a high-entropy session token and expire it. Set HttpOnly, Secure in HTTPS production, and an appropriate SameSite cookie policy.

Logout should revoke the server session as well as clear the browser cookie. Changing password or account status may require broader session revocation.

Authorize every operation

Use the authenticated user id in queries for listing, reading, updating, and deleting. Reject client-owned ownership fields. A public resource id is not permission.

Return a safe missing-resource response to another user instead of revealing whether their id exists. Test cross-user access for every endpoint, not only the UI.

Constrain input and side effects

Validate scalar types and allowed values. Build explicit writes rather than spreading untrusted input. Restrict URL schemes and avoid fetching user-supplied URLs unless necessary.

Cookie-authenticated writes need CSRF defenses. The reference workspace uses same-origin deployment, a strict Origin check on writes, JSON bodies, and SameSite cookies. If deployment becomes cross-origin, revisit the whole contract.

Worked example

```
const task = await tasks.findOne({_id: requestedId, owner: req.user.id});
if (!task) return res.status(404).json({error: "NOT_FOUND"});
// Never take owner from req.body or a user-controlled header.
```

Demonstrate understanding

Use two accounts and verify that lists, updates, and deletes do not cross ownership boundaries.

Primary reference: <https://cheatsheetseries.owasp.org/>

Testing debugging and delivery

Test observable contracts

Write expected results from the specification, including boundaries, duplicates, invalid values, and failures. Avoid computing expected output with the same algorithm being tested.

A fake database is appropriate for route contracts. Real database tests are necessary for indexes and atomic predicates. Browser tests are necessary for focus and realistic interaction.

Debug systematically

Reproduce the issue, reduce it to a small case, inspect the relevant layer, and write a regression test where it adds value. Change one hypothesis at a time.

Use network logs for request failures, browser traces for rendering, and database explain output for query work. A rewrite without diagnosis may preserve the underlying failure.

Make setup repeatable

Commit lockfiles, provide env examples without secrets, document database requirements, and run builds in CI. State which checks passed and which environments were unavailable.

Tests should fail when important behavior breaks. A project with only screenshots or syntax checks has not established authorization, persistence, or recovery correctness.

Worked example

```
import {test} from "node:test";
import assert from "node:assert/strict";
test("a stale edit cannot overwrite the newer task", async () => {
  // Arrange two requests at the same expected version.
  // Assert one succeeds, one conflicts, and data changes only once.
});
```

Demonstrate understanding

Add a regression test for a bug you can reproduce, then verify that reverting the fix makes the test fail.

Primary reference: <https://testing-library.com/docs/react-testing-library/intro/>

System design for product engineers

Clarify the contract

State the core use cases, scale estimates, latency target, consistency requirement, and operational constraints. Label estimates as assumptions. Choose a simple initial design and identify the first bottleneck.

For a URL shortener, ask about custom aliases, expiration, abuse, and analytics. For a feed, ask about ordering, private content, pagination, and freshness.

Design for partial failure

Use timeouts, bounded retries, and idempotency for operations that can repeat. Queues usually provide at-least-once processing under common configurations, so consumers must tolerate duplicates.

An outbox connects a database write to eventual event publication, but consumers still deduplicate. Monitor backlog and provide a failure-handling path.

Explain evolution

Caches need correct key scope and freshness rules. Replication changes failure and consistency behavior. Sharding and extra services add operational work.

Discuss a measurable trigger for each additional component. If no requirement needs a distributed design, a well-structured service and database may be the better starting point.

Worked example

```
// An idempotent write needs durable state, not only a process-local Set.  
// unique index: {owner: 1, operationId: 1}  
// Persist operation result with the business change under the chosen atomic boundary.
```

Demonstrate understanding

Draw a notification design, trace a crash between save and publish, and explain how the request eventually reaches a user without duplicate business effects.

Primary reference: <https://www.mongodb.com/docs/manual/core/transactions/>

TypeScript and explicit domain states

Represent valid states

Use discriminated unions when fields depend on a state. A success variant carries data; an error variant carries a message. This reduces impossible combinations compared with several independent booleans.

Prefer unknown for external data until validated. Static types describe your assumptions; runtime checks establish whether incoming data satisfies them.

Keep relationships in types

Generics connect input and output types without discarding specifics. Constraints express the capabilities required. Narrow values with checks rather than using broad type assertions.

Use strict compiler settings for a new typed project. Avoid any at API boundaries, and do not claim that a TypeScript migration makes an unvalidated server secure.

Practice migration

Migrate a small service or state reducer first. Keep runtime behavior stable, add precise types, and retain tests. Typed libraries can still deliver invalid data at runtime.

Explain where types help and where they do not: authorization, network failure, concurrency, and database constraints remain runtime concerns.

Worked example

```
type Remote<T> =
  | {status: "idle"}
  | {status: "loading"}
  | {status: "success"; data: T}
  | {status: "error"; message: string};
function summary(state: Remote<string[]>) {
  return state.status === "success" ? state.data.length : 0;
}
```

Demonstrate understanding

Port one API response parser and one React reducer to TypeScript with strict checking and no unchecked assertion of external data.

Primary reference: <https://www.typescriptlang.org/docs/>

Part 2 Difficulty based questions and answers

Html Css

Q001 Easy Why use a button instead of a clickable div?

A button has keyboard activation, focus behavior, a role, and a disabled state built in. A div needs those behaviors implemented separately. Use `type="button"` inside a form when the action should not submit. Test with Tab, Enter, and Space.

Follow-up: How would you make a toggle announce its state?

Q002 Easy What is the difference between an id and a class?

An id identifies one element in a document and can connect a label to a form control or a fragment link to a section. Classes group elements for styling and scripting. Duplicate ids make these relationships ambiguous. Prefer classes for reusable styles.

Follow-up: How does specificity differ?

Q003 Easy Explain the CSS box model.

An element has content, padding, border, and margin. With content-box, the declared width covers content; padding and borders add to the rendered width. With border-box they are included. Margins remain outside in either case.

Follow-up: Why can two vertical margins collapse?

Q004 Medium When would you choose Grid over Flexbox?

Grid manages tracks in two dimensions and is useful for page layouts or aligned card rows and columns. Flexbox distributes items along one main axis and works well for toolbars. A grid item can itself be a flex container; choose based on the alignment constraint.

Follow-up: How does `minmax(0,1fr)` prevent overflow?

Q005 Medium Why is z-index 999999 still behind a modal?

z-index compares elements inside their stacking context. A descendant cannot escape an ancestor context merely by increasing its own value. Inspect positioned ancestors, transforms, opacity, and isolation. Native dialog top-layer behavior is a separate mechanism.

Follow-up: Which properties create a stacking context?

Q006 Medium How do you build an accessible form?

Associate visible labels with controls, choose input types and autocomplete values, group related controls with fieldset and legend, and connect error text with `aria-describedby`. Preserve entered values after failure. Client validation improves feedback; the server validates independently.

Follow-up: Where should focus move after a failed submission?

Q007 Medium What does the cascade actually decide?

For declarations that apply to an element, the cascade considers origin and importance, cascade layers, specificity, and ordering, with additional rules for scoped styles. Specificity alone cannot explain every result. Inspect computed styles before adding !important.

Follow-up: What changes for important declarations in layers?

Q008 Hard How do you debug layout shift?

Record a browser performance trace and inspect layout-shift entries. Reserve space for media, avoid inserting late content above what users are reading, and review font loading. A shift caused by recent user interaction may be treated differently in the metric; measure the actual experience.

Follow-up: How do you reserve image space responsively?

Q009 Hard How should a modal manage focus?

On opening, move focus into the dialog to a useful target; keep keyboard navigation within the active modal and provide an accessible name. On close, return focus to the trigger if it remains usable. Prefer the native dialog API and test Escape and background interaction.

Follow-up: What happens if the trigger is removed?

Q010 Hard How do you make a table usable on a phone?

Keep header relationships and readable text. A horizontal scroll container can preserve a true data table; give users a clear scroll affordance. For a card view, repeat the labels and preserve reading order. Choose based on whether cross-row comparison is essential.

Follow-up: How would you test at 200 percent zoom?

Javascript

Q011 Easy What does const guarantee?

const prevents reassignment of the binding. It does not freeze the object it references. `const user = {name:"A"}; user.name = "B"` is valid. `Object.freeze` is shallow, so nested objects still need an explicit immutability strategy.

Follow-up: What is the temporal dead zone?

Q012 Easy Compare strict equality and Object.is.

Strict equality avoids type coercion, considers +0 and -0 equal, and considers NaN unequal to itself. `Object.is` considers NaN equal to itself and distinguishes signed zeros. Choose a comparison that matches the domain rather than assuming one is universally better.

Follow-up: What does a Set do with NaN?

Q013 Easy Explain a closure with a useful example.

A function retains access to the lexical environment where it was created. A counter factory can keep a private count and return an increment function. Two factory calls create separate environments. Closures are also why an old callback may see an earlier render value.

Follow-up: Can a closure retain a large object?

Q014 Medium Predict synchronous logs, a promise, and a timer.

In a normal browser script, synchronous code finishes first. Promise reaction jobs run at the microtask checkpoint before the next timer task. `setTimeout(fn,0)` makes a callback eligible later; it does not interrupt the current stack. Do not generalize this ordering to every Node phase.

Follow-up: Can recursive microtasks starve rendering?

Q015 Medium Why does spreading an object not deep clone it?

Object spread copies own enumerable properties into a new object. Nested object references still point to the same objects. `structuredClone` supports many structured values but cannot clone functions. For React state, copy only the path being changed or use an appropriate immutable update tool.

Follow-up: What happens to a Date in JSON serialization?

Q016 Medium Compare debounce and throttle.

Debounce waits for a quiet period and is useful for search inputs. Throttle limits execution frequency and is useful for continuous scroll work. Specify leading and trailing behavior, cancellation, and what happens to this and arguments; those are part of the function contract.

Follow-up: How would you flush the pending call?

Q017 Medium Compare Promise.all and Promise.allSettled.

`Promise.all` resolves with ordered results only when every input fulfills and rejects when an input rejects. It does not cancel other operations. `allSettled` waits for all inputs and reports each outcome. Bound concurrency for large input lists instead of starting thousands at once.

Follow-up: How can `AbortController` help cancellation?

Q018 Hard How does this behave in arrow functions?

An arrow function captures this lexically. A normal function gets this from the invocation form, such as `obj.method()`, and a detached method loses that receiver. `bind` creates a function with a fixed receiver. Arrow functions cannot be constructors.

Follow-up: Why can an arrow method be wrong on a prototype?

Q019 Hard How would you implement an LRU cache?

Use a Map for insertion order. On get, remove and reinsert the entry to mark it recent. On set, replace any existing entry and evict the first key when over capacity. Average operations are constant time under the usual hash-map model. Validate capacity and distinguish missing keys from stored undefined.

Follow-up: How would you add TTL without unbounded timers?

Q020 Hard What causes memory leaks in browser code?

A leak is memory retained longer than needed: global caches with no eviction, listeners on long-lived objects, timers holding closures, or subscriptions not removed. Compare heap snapshots after repeating a flow. Removing a DOM node alone does not release references held elsewhere.

Follow-up: How would you prove that a fix works?

React

Q021 Easy What is the difference between props and state?

Props are inputs supplied by a parent. State is component-owned memory that participates in rendering. Treat both as immutable snapshots. Derive values from existing inputs during render when possible instead of keeping duplicate state that can drift.

Follow-up: When should state be lifted?

Q022 Easy Why do list items need stable keys?

Keys tell React which sibling identity persists across renders. A stable domain id preserves the correct item state when ordering changes. Array indexes can assign state to the wrong row after insertion or sorting. Keys only need to be unique among siblings.

Follow-up: Can changing a key reset a form?

Q023 Easy Why does `setCount(count + 1)` twice often add only one?

Both calls can read the same state snapshot in the event handler. Functional updates `setCount(c => c + 1)` describe transitions and can be applied in sequence. The state variable in the current handler does not mutate immediately after a setter.

Follow-up: What does batching change?

Q024 Medium What belongs in `useEffect`?

Use an effect to synchronize with an external system such as a subscription, network request, or browser API. User actions usually belong in event handlers. Filtering an existing array can happen during render. Return cleanup for resources acquired by the effect.

Follow-up: Why might development run setup and cleanup again?

Q025 Medium How do you avoid stale search responses?

Associate the request with the current query, abort obsolete fetches during cleanup, and ignore responses from older requests. Cancellation can reduce work; an identity or generation check protects correctness even if an operation cannot be canceled. Keep loading and error state tied to the same request.

Follow-up: What happens when the user clears the query?

Q026 Medium When would you use a reducer?

A reducer makes related state transitions explicit, especially when several fields change together. Keep the reducer pure and represent actions using domain intent. It can make undo or optimistic transitions easier to inspect. A single simple boolean does not need a reducer.

Follow-up: Where do asynchronous operations belong?

Q027 Medium Does context prevent renders?

Context distributes a value. Consumers can render when the provided value changes, and normal parent rendering also matters. Split unrelated contexts and keep provider values stable where useful. Measure before adding memoization; context is not a complete server-data cache.

Follow-up: When would an external store help?

Q028 Hard How do you handle optimistic updates safely?

Apply a temporary local result while a request is pending. Use a mutation id or version so an earlier response cannot overwrite a later edit. On failure, restore the affected item or refetch authoritative state and show a retry path. Disabling duplicate submission simplifies some flows.

Follow-up: How do you handle a 409 version conflict?

Q029 Hard How do you improve a slow React list?

Measure whether the cost is fetching, rendering, layout, or large data processing. Use server pagination or windowing for large collections, stable keys, and appropriate state placement. Memoization helps only when avoided work exceeds comparison cost and props remain suitably stable.

Follow-up: How do you preserve accessibility in virtualization?

Q030 Hard Compare client components and server components.

Client components support state and browser interactions. Server components execute in a compatible server-rendering framework and can access server resources without shipping that implementation to the browser. A Vite client SPA does not gain server components by installing React.

Follow-up: How do you keep secrets out of client bundles?

Node Express

Q031 Easy What does Node add to JavaScript?

Node is a runtime with APIs for processes, files, networking, and other server tasks. Browser DOM APIs are not generally available. Nonblocking I/O lets a process handle many waiting operations, but synchronous JavaScript work still occupies the main execution thread.

Follow-up: Which APIs use the worker pool?

Q032 Easy What is Express middleware?

Middleware receives the request, response, and next function. It may finish the response or pass control onward. Ordering matters: JSON parsing before routes, authentication before protected handlers, and error handlers after routes. Calling next after sending a response can create double-response bugs.

Follow-up: How is error middleware identified?

Q033 Easy Why must input validation run on the server?

Clients can be modified or bypassed. Validate types, lengths, allowed values, and relationships at the API boundary. Use an allowlist when constructing database writes and return an actionable validation error. Database constraints remain necessary for races and persistent invariants.

Follow-up: Should you pass req.body directly into an update?

Q034 Medium How do Express 4 and Express 5 differ for async errors?

Express 5 forwards rejected promises returned from route handlers to error middleware. Express 4 handlers usually need an explicit wrapper or try/catch with next(error). Use the behavior of the installed major version and test a thrown asynchronous error.

Follow-up: Why should unexpected errors not expose stacks?

Q035 Medium When should you use a stream?

Streams process a sequence without loading the complete payload into memory. Respect backpressure when the receiver cannot keep up. pipeline coordinates completion and error handling. A stream helps large files; it does not make CPU-heavy parsing automatically inexpensive.

Follow-up: How does a client disconnect affect a response stream?

Q036 Medium How do you shut down an API gracefully?

Stop accepting new requests, let in-flight work finish within a deadline, close database and queue connections, and exit. Handle termination signals and readiness separately from liveness. Long-lived sockets require an explicit policy so shutdown cannot hang forever.

Follow-up: What happens during a rolling deployment?

Q037 Medium How do you diagnose a blocked event loop?

Correlate latency spikes with CPU profiles and event-loop delay metrics. Look for synchronous filesystem calls, large JSON work, catastrophic regexes, or expensive computations. Split or offload CPU work to worker threads and set payload limits. More async keywords cannot fix synchronous CPU work.

Follow-up: How would you benchmark a proposed fix?

Q038 Hard How should rate limits work across multiple instances?

Per-process limits are useful locally but do not give one consistent global budget. Use a shared store or gateway and choose a key, window algorithm, and outage policy. Be careful with forwarded IP headers: trust only known proxy configuration.

Follow-up: How would you limit by both account and IP?

Q039 Hard How do you safely retry an API request?

Retry transient failures with a bounded attempt count, timeout, exponential backoff, and jitter. Retrying a write can duplicate side effects, so use idempotency keys or a domain uniqueness constraint. A validation failure should not be retried automatically.

Follow-up: How do you distinguish timeout from definitive failure?

Q040 Hard What should structured API logs contain?

Include a request or trace id, route template, status, duration, and a safe error category. Redact passwords, authorization headers, session tokens, and sensitive personal data. Structured fields enable aggregation; logging every request body risks exposing data and increases cost.

Follow-up: How do you correlate logs across services?

Mongodb

Q041 Easy When should you embed rather than reference?

Embed bounded data that is read and updated with its parent. Reference independently owned entities or collections that grow without a useful bound. Model around access patterns and update invariants. MongoDB is flexible, but an application still needs schema and migration discipline.

Follow-up: How would you model comments on a popular post?

Q042 Easy What does an index do?

An index maintains an ordered access structure so queries can locate matches without scanning every document. It consumes storage and adds write work. Inspect explain output on realistic data to see whether the index supports the actual filter and sort.

Follow-up: Why does indexing every field hurt?

Q043 Easy Is Mongoose unique a validator?

unique declares an index intention. The database unique index enforces uniqueness and concurrent duplicate writes may produce E11000. Ensure the index exists and translate the conflict into an appropriate API response. An earlier findOne check alone is race-prone.

Follow-up: What about existing duplicates before adding the index?

Q044 Medium Explain a compound index.

A compound index orders multiple fields. Its prefixes and key order affect which queries can use it efficiently. For a user-owned feed, {owner:1, createdAt:-1, _id:-1} can support a constrained ordered listing. Check filters, sort, and scanned keys rather than choosing an index from intuition alone.

Follow-up: How does a range predicate change your index choice?

Q045 Medium Compare offset and cursor pagination.

Offset pagination is simple and allows page numbers, but large offsets do extra work and concurrent inserts can shift boundaries. Cursor pagination seeks after a stable sort tuple and is useful for feeds. Include a tie-breaker such as id and validate cursor format and scope.

Follow-up: Can users jump to page 50 with a cursor?

Q046 Medium When do you need a transaction?

Single-document writes are atomic. A transaction is useful when a business invariant spans multiple documents and all changes must commit together. Transactions require a supported deployment, typically a replica set or sharded cluster. They add coordination cost and still need appropriate retry handling.

Follow-up: Can embedding remove the transaction requirement?

Q047 Medium What does lean do in Mongoose?

lean returns plain objects instead of hydrated documents for a query. This can reduce memory and processing for read-only paths. Document methods and standard hydration behavior are absent, so check getters, virtuals, and serialization expectations before enabling it.

Follow-up: Would you use lean for a document you intend to save?

Q048 Hard How do you prevent selling the same last item twice?

Use an atomic conditional update whose filter includes available stock, with an increment or decrement in the same operation. A read-then-write sequence races. If payment and reservation span services, design expiration, idempotency, and compensation rather than assuming one database transaction covers them.

Follow-up: How would you release an expired reservation?

Q049 Hard How do you detect and prevent lost updates?

Include an expected version in the update predicate and increment it atomically. If no document matches, distinguish missing or unauthorized data from an owned version conflict without revealing another user's data. Tell the client to reload or resolve the conflict.

Follow-up: Can a timestamp be a reliable version token?

Q050 Hard What can make an aggregation slow?

Large input cardinality, early fan-out, expensive sorts, broad lookups, and insufficiently selective predicates can cause work and memory pressure. Use explain, realistic sample data, suitable indexes, and early filters where semantically valid. Avoid returning unbounded results to the client.

Follow-up: When would you precompute a summary?

Security Api

Q051 Easy Compare authentication and authorization.

Authentication establishes who the caller is. Authorization decides whether that caller may perform an action on a resource. Every protected database operation should enforce ownership or role scope. A hidden UI button is not an authorization check.

Follow-up: Why is checking only the route id insufficient?

Q052 Easy Why hash passwords rather than encrypt them?

A password verifier should not require recovering the original password. Use a password hashing function with a unique salt and appropriate work parameters. Store the encoded parameters for upgrades. Never use a plain fast digest alone or log the password during debugging.

Follow-up: How do you migrate work factors?

Q053 Easy What does CORS protect?

CORS controls browser access to cross-origin responses. It is not API authentication and does not stop server clients from making requests. Some cross-origin requests can still be sent even when reading the response is blocked. Cookie-based writes need a deliberate CSRF strategy.

Follow-up: Why can CORS alone fail to prevent CSRF?

Q054 Medium Where should a browser session token live?

An HttpOnly cookie prevents JavaScript from reading the token and can reduce token theft through XSS. Use Secure over HTTPS and an appropriate SameSite setting. Cookie authentication also needs CSRF defenses and revocation. Browser storage tradeoffs should be discussed with the threat model.

Follow-up: Does HttpOnly stop malicious scripts making requests?

Q055 Medium Why are JWTs not encrypted by default?

A usual signed JWT protects integrity, not confidentiality. Its payload is encoded and readable. Verify signature, allowed algorithms, expiration, issuer, and audience as appropriate. Revocation and rotation require a design; long-lived bearer tokens increase exposure.

Follow-up: How would you revoke one compromised session?

Q056 Medium How do you prevent NoSQL injection?

Construct filters from validated scalar fields rather than accepting arbitrary objects from the client. Reject unexpected operators and properties, limit lengths, and use explicit update fields. A schema library helps at the boundary but cannot repair a deliberately unsafe filter construction.

Follow-up: What is mass assignment?

Q057 Medium What makes a REST error useful?

Use a consistent status and a machine-readable error code with a safe human message. Include a correlation id for unexpected errors and field-level detail for validation failures. Avoid leaking stack traces, password hashes, or resource existence across authorization boundaries.

Follow-up: When would you return 409 rather than 400?

Q058 Hard How do you stop an SSRF attack?

Treat server-side fetching of user URLs as a privileged operation. Restrict destinations, schemes, redirects, and resolved IPs, block internal networks, and use egress controls and timeouts. Validation before a redirect or DNS change is not enough. A bookmark app can simply store URLs without fetching them.

Follow-up: How would you handle DNS rebinding?

Q059 Hard How do you make a payment webhook idempotent?

Verify the provider signature against the expected raw body, persist a unique event or operation id, and apply state changes atomically with that record. Handle out-of-order events and delayed retries. Return success only under a defined durable handling contract.

Follow-up: Why is an in-memory Set insufficient?

Q060 Hard How do you make file uploads safer?

Limit size and count, validate expected types using content as well as names, generate server filenames, store outside executable paths, and isolate processing. Apply malware scanning where relevant and authorize access to private files. Streaming and timeouts protect resources.

Follow-up: Can MIME type from the client be trusted?

Testing Tooling

Q061 Easy What should a unit test verify?

A unit test checks an observable behavior of a small unit with controlled inputs. Cover boundaries and errors, not just the happy path. Tests that repeat the implementation algorithm can pass despite the same bug. Use explicit expected outcomes or an independent model.

Follow-up: What makes a test brittle?

Q062 Easy What belongs in an integration test?

Check the contract between real cooperating pieces, such as HTTP parsing, authorization, service logic, and persistence. An in-memory repository is useful for route behavior but cannot establish MongoDB index or atomicity behavior. Label that distinction in the test report.

Follow-up: Which dependencies should remain real?

Q063 Easy Why commit package-lock.json?

The lockfile records the resolved dependency graph and integrity information. npm ci installs from it without rewriting resolutions and fails if the manifest disagrees. A lockfile improves reproducibility but does not eliminate vulnerabilities or runtime incompatibility.

Follow-up: How do you update dependencies deliberately?

Q064 Medium How should React tests select elements?

Prefer accessible roles and names because they reflect how users interact. Test an action and its visible result. Avoid selectors coupled to internal component structure unless necessary. Add browser tests for focus, layout, navigation, and behaviors that a simulated DOM does not model well.

Follow-up: When is a test id justified?

Q065 Medium How do fake timers help debounce tests?

Advance a controlled clock to assert no call before the quiet period, a call after it, and cancellation behavior. Verify arguments and this preservation. Avoid arbitrary real sleeps that make the suite slow and flaky. Restore the clock after the test.

Follow-up: How do microtasks interact with your test clock?

Q066 Medium How do you test authorization?

Create two users and resources for each. Attempt every read, update, and delete as the other user and as an unauthenticated caller. Verify both status codes and unchanged data. Include list endpoints because they can leak records even if item routes are protected.

Follow-up: How do you test role escalation?

Q067 Medium What makes a useful CI pipeline?

Run deterministic checks for documentation, unit and API tests, and production builds. Add database integration tests when persistence invariants matter. Keep installation reproducible with lockfiles and grant minimal permissions. A green build only proves the checks that actually ran.

Follow-up: Which checks need secrets?

Q068 Hard How do you test concurrency bugs?

Run competing operations against the actual persistence layer, start them together, and assert the invariant after both complete. For stock, total successful reservations must not exceed initial quantity. For version edits, only one write at the same expected version should succeed.

Follow-up: Why may a fake repository hide the race?

Q069 Hard What does test coverage not tell you?

Coverage reports which code was executed, not whether assertions establish correctness. A high percentage can coexist with missing authorization or boundary tests. Review risk-based cases, mutation behavior, and failures before using coverage as a quality proxy.

Follow-up: How can mutation testing expose weak assertions?

Q070 Hard How do you test a production build?

Build with the actual toolchain, serve the generated assets, and exercise critical flows against the intended API mode. Check base paths, missing assets, client routing fallback, and environment variables. Development success does not prove deploy-time settings work.

Follow-up: Which variables become public in a client bundle?

System Design

Q071 Easy How should a design interview start?

Clarify users, core operations, scale assumptions, correctness needs, and constraints. State which numbers are hypothetical. Sketch the simplest design that meets those requirements, then discuss where it fails. Listing technologies before the problem is clear makes tradeoffs difficult to defend.

Follow-up: Which requirement would change your design most?

Q072 Easy Compare latency and throughput.

Latency is time per operation; throughput is operations completed per unit time. Increasing concurrency can improve throughput until a bottleneck is saturated, while making latency worse. Report distributions such as p95 and p99, not just averages, and include failures.

Follow-up: What does queueing do near saturation?

Q073 Easy What can a cache get wrong?

A cache can serve stale data, expose data across users if keyed incorrectly, or overload the origin when many entries expire. Define freshness, key scope, capacity, and invalidation. For private responses, authorization and tenant identity remain part of the contract.

Follow-up: What is a cache stampede?

Q074 Medium How would you design notifications?

Persist notification intent or use an outbox with the business write, then let workers deliver asynchronously. Track attempts and unique operation ids, use bounded retries, and separate delivery from user read status. At-least-once delivery requires idempotent consumers.

Follow-up: How do you prevent duplicate emails?

Q075 Medium When would you use a queue?

Use a queue for work that can happen after the request, such as report generation. Define acknowledgment, retry, ordering, capacity, and dead-letter handling. A queue moves and buffers work; it does not remove the need to handle overload or failures.

Follow-up: How would the user see job progress?

Q076 Medium How do you scope a multitenant API?

Establish the tenant from trusted authentication or verified membership. Include tenant scope in every relevant query, index, cache key, and background job. Reject client attempts to assign ownership. Test with multiple tenants and cover administrative paths.

Follow-up: How would you model shared resources?

Q077 Medium How do you design a search endpoint?

Start with exact requirements: prefix, substring, ranking, and dataset size. Bound query length and result count, choose an index-supported strategy, and measure. Escaped regex may protect correctness but still scan many records. A dedicated search engine adds operations and consistency tradeoffs.

Follow-up: When is a Mongo text index enough?

Q078 Hard Explain an outbox pattern.

In one database transaction, persist both the business change and an event record. A publisher delivers unpublished records and marks progress. Delivery can repeat if a crash occurs between publishing and marking, so downstream consumers still deduplicate. Monitor backlog and failed records.

Follow-up: How do you preserve per-entity ordering?

Q079 Hard How do you design a rate limiter?

Choose a policy such as token bucket based on burst and sustained-rate requirements. Update shared state atomically, define keys and quotas, and make an explicit store-outage policy. Measure fairness and provide retry guidance. A fixed window can permit bursts around its boundary.

Follow-up: How would you support per-plan limits?

Q080 Hard How do you approach a region outage?

Clarify recovery time and acceptable data loss. Discuss replication lag, routing, backups, failover authority, and writes during partition. Exercise recovery procedures rather than assuming backups are restorable. Consistency and availability choices depend on the operation and business invariant.

Follow-up: How would you avoid two active writers?

Dsa Machine Coding

Q081 Easy How do you solve Two Sum in JavaScript?

Scan once with a Map from value to earlier index. For each value, check whether its complement is already present before inserting it. This avoids using one item twice. Expected time is $O(n)$, space $O(n)$. Test duplicate values and no-solution behavior.

Follow-up: What if the input is sorted?

Q082 Easy When is a sliding window appropriate?

A window works when adjacent ranges can be updated incrementally and a valid shrinking or expansion rule exists. For longest substring without repetition, track counts or latest indexes. For arbitrary sums with negative values, a simple increasing-sum window may be invalid.

Follow-up: Why do negative numbers break some window rules?

Q083 Easy How do you begin a machine coding round?

Clarify the required behavior and data contract, write a short acceptance checklist, and deliver one complete interaction early. Use semantic controls and separate core state logic from rendering. Reserve time for failure states and a demo instead of polishing before the main flow works.

Follow-up: What would you cut with ten minutes left?

Q084 Medium How do you implement a keyboard autocomplete?

Track query, matching results, active option, and expanded state. Support arrows, Enter, Escape, and appropriate combobox relationships. Avoid stale async responses and preserve focus on the input. Test an empty list and selection after the list changes.

Follow-up: How do you announce the result count?

Q085 Medium How do you flatten a nested tree?

Traverse recursively or with an explicit stack and carry the parent path or depth. State whether input order must be preserved and whether cycles are possible. Recursion uses stack space proportional to depth; an explicit stack avoids call-stack overflow for deeply nested inputs.

Follow-up: How do you handle duplicate ids?

Q086 Medium How do you implement undo in a task list?

Represent edits as commands or retain bounded prior states. Decide whether undo applies to local unsaved changes or to server-persisted operations. Give deleted items a stable id and protect against later conflicting changes. Announce the undo action and expiration clearly.

Follow-up: What happens after a page reload?

Q087 Medium How would you test a sortable data table?

Test text, numeric, and date comparisons, ties, direction changes, and mutation safety. Use a stable tie-breaker where order matters. Keep sort state explicit and announce the active sort with aria-sort. Test pagination interactions because sorting only the visible page can mislead users.

Follow-up: What is the locale policy?

Q088 Hard How do you build a concurrency-limited promise pool?

Keep at most k workers. Each worker claims a next index and awaits one task before claiming another. Store results at the original index and define error behavior. A fail-fast pool must still account for already-running tasks; a limit is not automatic cancellation.

Follow-up: What if one task never resolves?

Q089 Hard How do you implement an event emitter?

Map each event name to listeners, return an unsubscribe function, and decide how duplicate listeners behave. Iterate a snapshot so a listener removing another does not unexpectedly alter the current dispatch. Define exception and once semantics explicitly.

Follow-up: Should newly added listeners fire during current emission?

Q090 Hard How do you defend your coding solution?

Explain the state model, a core invariant, the expected complexity, and one rejected alternative. Demonstrate a boundary input and failure recovery. Identify what remains incomplete and how you would test it. Unsupported claims of production readiness weaken an otherwise good implementation.

Follow-up: What did you intentionally omit?

Typescript Career

Q091 Easy What does TypeScript guarantee at runtime?

Type annotations are erased and do not validate network responses at runtime. TypeScript catches many static mistakes when types are accurate. Parse unknown external values with runtime checks before trusting them. Avoid asserting a type merely to silence a compiler error.

Follow-up: Why is unknown safer than any?

Q092 Easy Compare an interface and a type alias.

Both can describe object shapes. Interfaces support declaration merging and extension; aliases also represent unions, primitives, and other type expressions. Use a consistent convention and choose the feature required by the design. Neither creates runtime data or validation.

Follow-up: How would you represent loading, success, and error?

Q093 Easy How should you explain a portfolio project?

State the user problem, your contribution, the key design choice, and a demonstrable result. Distinguish measurements from assumptions. Point to working source, tests, and setup instructions. Do not claim users, revenue, or latency numbers that you did not measure.

Follow-up: Which part would you redesign?

Q094 Medium What is a discriminated union useful for?

A shared literal field lets the compiler narrow a value to a particular variant. A request state can have idle, loading, success with data, or error with a message. This prevents combinations such as success without data that arise from independent booleans.

Follow-up: How do you check exhaustiveness with never?

Q095 Medium How do generics improve reusable functions?

A generic expresses a relationship between input and output types, such as `identity<T>(value:T):T`. Constraints limit allowed inputs without discarding their specific types. A generic parameter used only once may not add a useful relationship.

Follow-up: How would you type a key-based lookup?

Q096 Medium How should you answer a behavioral failure question?

Use a specific situation, your responsibility, the action you actually took, and the outcome or lesson. Own your contribution without blaming others or inventing a perfect ending. Describe what changed afterward and what evidence supports the result.

Follow-up: What would you do differently now?

Q097 Medium How should you prepare for an unfamiliar startup?

Read its product and role description, identify the workflows users rely on, and practice a relevant small build. Confirm interview format with the recruiter. Prepare questions about ownership, quality, operations, and constraints rather than memorizing rumored company question lists.

Follow-up: What would you ask about on-call work?

Q098 Hard How do you evaluate a technical tradeoff?

State the requirement, compare alternatives using correctness, complexity, delivery time, and operations, then explain what evidence would change your choice. Name the downside of your selected option and a migration path. An absolute rule usually hides an unstated assumption.

Follow-up: When does your chosen design stop working?

Q099 Hard How do you prepare for a project deep dive?

Trace one request from UI to persistence, explain validation and authorization, and rehearse a failure path. Know the indexes, concurrency behavior, tests, and deployment requirements of your own code. Be able to change one feature during the discussion.

Follow-up: Can you show a failing test and its fix?

Q100 Hard How do you close knowledge gaps without delaying applications forever?

Use a concrete diagnostic: solve a JS exercise, complete a timed UI, build a protected API, and defend a project. Track observed weaknesses and revisit them with retrieval practice. Apply when you can demonstrate the role's core skills while continuing targeted practice.

Follow-up: Which evidence shows improvement over the last two weeks?

Part 3 Scenario diagnosis

SC01 Search results jump backward while typing

Evidence: Throttle the network and type three queries quickly. Record each request id and response time.

Likely cause: An earlier request completes after a later request and overwrites the visible state.

Fix: Abort obsolete requests and guard completion with a request generation. Clearing the query also invalidates work.

Verification: Resolve mocked requests in reverse order; the final UI must match the newest query.

Common mistake: Debounce reduces requests but does not guarantee ordering.

SC02 A timer always sees the first counter value

Evidence: Log the render value and callback value and inspect effect dependencies.

Likely cause: A callback captured a prior render snapshot.

Fix: Use a functional state update when only the previous value is needed, or resubscribe with correct dependencies. Use a ref only when a current mutable value is truly required.

Verification: Advance timers after several increments and verify current behavior and cleanup.

Common mistake: Suppressing the dependency warning hides the stale closure.

SC03 Editing a sorted list changes the wrong row

Evidence: Type into a row, then sort or insert at the front.

Likely cause: Index keys give the row component a different domain item while keeping state.

Fix: Key by stable item id and decide whether edits belong in row-local drafts or a keyed parent model.

Verification: Edit one id and reorder; the draft remains attached to that id.

Common mistake: Random keys force remounts and lose all drafts.

SC04 An optimistic task update overwrites a newer edit

Evidence: Send two edits from separate tabs at the same version.

Likely cause: The server accepts writes without a concurrency condition.

Fix: Require expectedVersion, update atomically, and return an owned conflict. On conflict, refresh and let the user reconcile.

Verification: Against real MongoDB, run concurrent updates and assert exactly one success.

Common mistake: The UI alone cannot serialize other devices or processes.

SC05 A form looks fine but cannot be completed with a keyboard

Evidence: Use Tab and Shift+Tab without the mouse. Inspect role and label relationships.

Likely cause: Clickable divs, missing labels, or focus loss break interaction.

Fix: Replace controls with native buttons and inputs, restore focus after dialogs, and associate errors with controls.

Verification: Complete the flow with keyboard and inspect accessible names.

Common mistake: Adding tabIndex alone does not supply activation or semantics.

SC06 A card pushes a grid wider than the viewport

Evidence: Test a long unbroken title and inspect the overflowing box.

Likely cause: An intrinsic minimum or fixed width prevents shrinking.

Fix: Use minmax(0,1fr), min-width:0, wrapping, and constrained media where the content contract permits.

Verification: Check 320px width and 200 percent zoom with long input.

Common mistake: Hiding overflow can conceal required information.

SC07 All requests slow down during report generation

Evidence: Correlate CPU profiles and event-loop delay with report requests.

Likely cause: Synchronous CPU work blocks the request-handling thread.

Fix: Move expensive work to a worker or job queue, bound workload, and report asynchronous progress.

Verification: Load test with concurrent small requests and a report; compare tail latency.

Common mistake: Making the function async does not offload computation.

SC08 Memory grows on every page visit

Evidence: Repeat the flow, force comparable heap snapshots, and inspect retaining paths.

Likely cause: A long-lived listener or cache retains per-request or per-page data.

Fix: Unsubscribe, clear timers, or bound and evict cache entries; verify the actual retaining path.

Verification: Repeat the workload after the fix and compare retained objects.

Common mistake: A single lower heap reading does not prove leak removal.

SC09 User A can delete User B's task by changing an id

Evidence: Create two users and attempt the delete as A using B's id.

Likely cause: The database filter lacks authenticated ownership.

Fix: Include owner in every item and list query. Ignore client ownership fields and return a safe missing response.

Verification: Read, update, delete, and list isolation tests all preserve B's data.

Common mistake: Hiding ids or changing to UUIDs does not authorize access.

SC10 Cross-site form submission changes account settings

Evidence: Inspect cookie settings, content type, Origin checks, and CSRF policy.

Likely cause: Cookie authentication is accepted without a coherent write-origin defense.

Fix: Use a defined CSRF strategy, appropriate SameSite cookies, and same-origin enforcement for the reference deployment. Reassess cross-origin requirements.

Verification: Send missing and foreign Origin writes; both must fail under the chosen contract.

Common mistake: CORS response restrictions are not a complete CSRF policy.

SC11 Two requests both purchase the last item

Evidence: Run two overlapping reservations against stock=1.

Likely cause: Each request reads stock before an unconditional decrement.

Fix: Use one atomic predicate with stock $\geq$ requested amount and decrement in that write. Coordinate reservation expiration and payment separately.

Verification: At most one reservation succeeds; stock remains nonnegative.

Common mistake: A process-local mutex does not protect multiple instances.

SC12 A uniqueness check passes but duplicate registration fails

Evidence: Send simultaneous registrations and inspect database index errors.

Likely cause: Both prechecks pass before either inserts.

Fix: Enforce the unique index and map duplicate-key errors to a safe conflict response. Prechecks may improve feedback but are not the invariant.

Verification: Exactly one insert succeeds under concurrency.

Common mistake: Removing the index to avoid E11000 removes protection.

SC13 The feed skips or repeats items during new inserts

Evidence: Compare offset pages while inserting earlier records.

Likely cause: Offset boundaries shift and timestamp ties are not uniquely ordered.

Fix: Use a cursor containing the last stable sort tuple, with an id tie-breaker and validated scope.

Verification: Insert between page requests and verify stable traversal of the documented contract.

Common mistake: Cursor pagination is not automatically a snapshot of all data.

SC14 A small database query becomes slow at scale

Evidence: Run explain on representative cardinality and compare examined versus returned records.

Likely cause: The query scans too much or sorts outside a useful index.

Fix: Design an index for real equality, sort, and range requirements; bound responses and measure before and after.

Verification: Document query plans and latency under the same dataset.

Common mistake: A faster run on ten sample rows is weak evidence.

SC15 A JWT includes a password hash

Evidence: Decode a test token and inspect only safe nonproduction data.

Likely cause: Sensitive data was put in a readable signed payload.

Fix: Remove sensitive claims, rotate affected credentials or sessions as required, and restrict claims to the minimum needed.

Verification: Assert serialized tokens and API responses never contain secret fields.

Common mistake: Signing a payload does not encrypt it.

SC16 A bookmark preview reads an internal admin endpoint

Evidence: Inspect the server fetch destination, redirects, and resolved addresses.

Likely cause: The application performs unrestricted server-side URL fetching.

Fix: Avoid server fetching in a simple bookmark project. If previews are required, isolate egress and validate each destination and redirect.

Verification: Reject internal destinations in a controlled test environment.

Common mistake: Checking only that the string begins with https is insufficient.

SC17 The payment provider retries and a user is charged twice

Evidence: Correlate operation ids, webhook ids, and durable records.

Likely cause: The handler assumes one delivery or retries a non-idempotent operation.

Fix: Use provider-supported idempotency and a durable unique operation record with the chosen atomic boundary. Verify webhook signatures.

Verification: Replaying an event does not repeat the business effect.

Common mistake: An in-memory deduplication map disappears on restart.

SC18 The task was saved but its notification was lost

Evidence: Trace a crash immediately after the database commit and before queue publish.

Likely cause: Two separate writes do not share a durable handoff.

Fix: Use an outbox persisted with the business change and a retrying publisher. Make consumers idempotent.

Verification: Crash at each boundary; the event eventually publishes without duplicate business effects.

Common mistake: An outbox does not guarantee exactly-once transport.

SC19 Tests pass but the browser receives HTML for an API route

Evidence: Serve the built app and inspect content type on the API response.

Likely cause: An SPA fallback or proxy is matching an API path incorrectly.

Fix: Mount API routes first, return API 404s for unknown API paths, and restrict the SPA fallback to page navigation.

Verification: Test an unknown API path and a client route against the production server.

Common mistake: Development proxy success does not prove production routing.

SC20 A deployment installs different package versions

Evidence: Compare lockfile, manifest, runtime, and install command.

Likely cause: The build resolves floating versions or ignores the lockfile.

Fix: Commit the dependency lock, use npm ci, and declare compatible runtime versions. Update deliberately through a tested change.

Verification: A clean CI install produces the same resolved graph.

Common mistake: A lockfile does not validate all environments or dependency safety.

SC21 Rate limits disappear when another instance serves the request

Evidence: Route the same caller to multiple instances and compare counters.

Likely cause: Counters exist separately in process memory.

Fix: Use a shared limiter or gateway for distributed enforcement, with explicit outage and key policies.

Verification: A caller receives one global budget across instances.

Common mistake: A trusted proxy must be configured before using forwarded IPs.

SC22 Shutdown drops active requests

Evidence: Send a slow request and terminate the server in a controlled environment.

Likely cause: The process exits before requests and connections drain.

Fix: Stop accepting connections, drain within a deadline, close dependencies, and handle stuck sockets deliberately.

Verification: The slow request finishes or follows a documented bounded failure policy.

Common mistake: Waiting indefinitely can block deployment forever.

SC23 A typed API returns a string where a number is expected

Evidence: Inspect the incoming payload before an assertion or cast.

Likely cause: Type annotations were treated as runtime validation.

Fix: Parse unknown input at the boundary and reject invalid types. Preserve static types after successful validation.

Verification: An incorrect payload produces a validation error rather than silent arithmetic coercion.

Common mistake: Using as number changes no runtime value.

SC24 The interviewer asks for a tradeoff you cannot explain

Evidence: Trace a real feature and identify the requirement that led to the design.

Likely cause: The project was copied without understanding its constraints.

Fix: Implement a small extension yourself, document an alternative, and measure or test the deciding behavior.

Verification: Explain one invariant, demonstrate a failure path, and justify a different choice under a changed requirement.

Common mistake: Memorized slogans are not evidence of design judgment.

Part 4 Machine coding approaches

Attempt a build before reading its approach. Times are suggested limits. Score required behavior 30, boundaries and recovery 20, state and API design 15, accessibility 15, verification 10, and explanation 10.

MC01 Accessible accordion

Easy | 45 minutes | html-css

Required behavior: Multiple sections can open; buttons expose aria-expanded and controls ids.

Boundary checks: None expanded, one expanded, multiple expanded, keyboard activation.

Solution approach: Use a Set of section ids; render state through native buttons; keep content mounted if its state must persist.

MC02 Todo with undo

Easy | 45 minutes | javascript

Required behavior: Create, complete, filter, delete, and undo the most recent delete; ids are stable.

Boundary checks: Whitespace title, duplicate titles, delete under active filter, undo twice.

Solution approach: Use normalized ids and a pure reducer; keep one bounded deleted item for the stated contract.

MC03 Debounced search

Medium | 60 minutes | react

Required behavior: Search after 300ms quiet time; render loading, empty, error; latest query wins.

Boundary checks: Out-of-order responses, clearing query, unmount, retry after failure.

Solution approach: Debounce input and independently guard request identity; abort on cleanup; preserve query on retry.

MC04 Sortable paginated table

Medium | 75 minutes | react

Required behavior: Sort full dataset by name or numeric amount; page after sorting; preserve stable ids.

Boundary checks: Ties, zero, negative numbers, last page after deleting, empty data.

Solution approach: Keep sort and page as state; derive sorted and sliced rows; reset invalid pages; use aria-sort.

MC05 Modal with focus return

Medium | 60 minutes | html-css

Required behavior: Open from a trigger, accessible title, Escape close, focus return, background interaction blocked.

Boundary checks: Repeated open and close, trigger removed, long dialog, keyboard-only use.

Solution approach: Use native dialog and showModal; remember the invoking element; select a useful initial focus target.

MC06 Shopping cart

Medium | 75 minutes | react

Required behavior: Add items, change quantities 1..99, remove, show integer-cent totals, prevent duplicate submit.

Boundary checks: Stock changes, invalid quantity, empty cart, zero-price item.

Solution approach: Key lines by product id; derive totals; server must validate actual price and stock on checkout.

MC07 Nested file explorer

Medium | 90 minutes | react

Required behavior: Expand folders, select a file, rename and create unique sibling names.

Boundary checks: Deep nesting, empty folder, duplicate names, rename root policy.

Solution approach: Use ids independent of names; recurse for presentation and keep expanded ids separately.

MC08 Multi-step form

Medium | 75 minutes | react

Required behavior: Three steps, per-step validation, back preserves drafts, final submit once.

Boundary checks: Server error, navigating back, missing fields, keyboard submission.

Solution approach: One authoritative draft and explicit step state; validate transitions and retain data on errors.

MC09 Keyboard autocomplete

Hard | 90 minutes | react

Required behavior: Arrow navigation, Enter selection, Escape collapse, combobox semantics, latest async query wins.

Boundary checks: No matches, IME composition, changed list during selection, blur before click.

Solution approach: Keep active index valid as results change; use aria-activedescendant and pointer handling without stealing input focus.

MC10 Kanban with conflict recovery

Hard | 90 minutes | react

Required behavior: Move tasks among columns using buttons; persist versioned writes; recover on conflict.

Boundary checks: Two tabs, failed write, delete while moving, empty column.

Solution approach: Separate local pending state from server state; expected version belongs to each mutation; refetch after conflict.

MC11 Promise pool

Hard | 75 minutes | javascript

Required behavior: Run at most k task factories, ordered results, explicit failure policy, invalid limit rejected.

Boundary checks: Empty tasks, rejection, k larger than input, synchronous throw.

Solution approach: Workers claim indexes and await one task each; store outcome by original index; account for tasks already running.

MC12 LRU with TTL extension

Hard | 75 minutes | javascript

Required behavior: Constant-average-time get and set, capacity eviction, optional injected clock for TTL.

Boundary checks: Existing key update, stored undefined, capacity=1, invalid capacity, expiry boundary.

Solution approach: Use Map order for recency and distinguish has from get; check expiry lazily and bound storage.

MC13 Protected CRUD API

Medium | 90 minutes | node-express

Required behavior: Validate input, authenticate, scope reads and writes by owner, bound list size.

Boundary checks: Unauthenticated access, cross-user ids, unknown properties, malformed body.

Solution approach: Build filters from validated fields and trusted identity; use one safe error mapper and route integration tests.

MC14 Inventory reservation API

Hard | 120 minutes | mongodb

Required behavior: Reserve stock atomically, operation id deduplication, expiration policy, no negative stock.

Boundary checks: Two last-item requests, repeated operation id, invalid quantity, expiration races.

Solution approach: Choose an atomic data model or transaction; persist deduplication with the business effect; test against real MongoDB.

MC15 Notification center

Hard | 90 minutes | react

Required behavior: Unread count, mark read, filter, pagination, retry errors, accessible announcements.

Boundary checks: Duplicate pages, repeated mark-read, newest item during paging, offline.

Solution approach: Normalize records by id; treat mark-read as idempotent; keep cursor and request state explicit.

MC16 Expense dashboard

Hard | 120 minutes | react

Required behavior: Add dated integer-cent expenses, filter category, total authoritative data, authenticated persistence.

Boundary checks: Invalid decimal input, date overflow, empty filter, cross-user access.

Solution approach: Parse money exactly, use validated calendar dates, derive totals consistently, and test ownership and amount boundaries.

Part 5 External learning references

Links were reviewed on 2026-10-08. A successful retrieval does not establish unchanged prices or hiring processes. Consult the repository catalog for partial checks.

MDN learning curriculum

Web fundamentals | Beginner | Free

Start with core modules; recreate each exercise without looking.

https://developer.mozilla.org/en-US/docs/Learn_web_development

web.dev courses

HTML CSS JS | Beginner to advanced | Free

Use HTML, CSS, forms, accessibility, and performance tracks.

<https://web.dev/learn>

JavaScript.info

JavaScript | Beginner to advanced | Free web; optional paid ebooks

Work through language, DOM, and asynchronous exercises.

<https://javascript.info/>

Full Stack Open

Full stack | Intermediate | Free

Complete React and Node exercises after JavaScript fundamentals.

<https://fullstackopen.com/en/>

The Odin Project JavaScript path

Full stack | Beginner to intermediate | Free

Use project assignments for independent builds.

<https://www.theodinproject.com/paths/full-stack-javascript>

React Learn

React | Beginner to advanced | Free

Build the examples and explain state transitions aloud.

<https://react.dev/learn>

You Might Not Need an Effect

React | Intermediate | Free

Review each effect in your project and justify its external synchronization.

<https://react.dev/learn/you-might-not-need-an-effect>

Node Learn

Node | Beginner to advanced | Free

Study runtime behavior alongside a small API.

<https://nodejs.org/en/learn/getting-started/introduction-to-nodejs>

Express installation and guides

Express | Beginner to intermediate | Free

Check the installed major version before following older tutorials.

<https://expressjs.com/en/starter/installing.html>

MongoDB manual getting started

MongoDB | Beginner | Free docs; hosted usage may cost

Practice CRUD on a local or test database.

<https://www.mongodb.com/docs/manual/tutorial/getting-started/>

Atomicity and transactions

MongoDB | Advanced | Free

Test concurrent writes rather than relying on a diagram.

<https://www.mongodb.com/docs/manual/core/write-operations-atomicity/>

Mongoose schemas guide

Mongoose | Intermediate | Free

Compare validation, indexes, and update behavior.

<https://mongoosejs.com/docs/guide.html>

TypeScript handbook

TypeScript | Beginner to advanced | Free

Port one reducer and validate external data at runtime.

<https://www.typescriptlang.org/docs/>

React Testing Library

Testing | Intermediate | Free

Test visible outcomes through accessible roles.

<https://testing-library.com/docs/react-testing-library/intro/>

Playwright documentation

Testing | Intermediate | Free

Test the built app and keyboard workflows.

<https://playwright.dev/docs/intro>

Vite guide

Build tools | Beginner to intermediate | Free

Understand development proxy versus production hosting.

<https://vite.dev/guide/>

OWASP Cheat Sheet Series

Security | Intermediate to advanced | Free

Use targeted authentication, session, CSRF, and input-validation references.

<https://cheatsheetseries.owasp.org/>

WAI tutorials

Accessibility | Beginner to advanced | Free

Review forms, images, tables, and page structure.

<https://www.w3.org/WAI/tutorials/>

Pro Git book

Git | Beginner to advanced | Free

Practice branches, conflict resolution, and recovery in a sandbox repository.

<https://git-scm.com/book/en/v2>

OpenAPI specification

API contracts | Intermediate | Free

Write and validate a small API contract.

<https://spec.openapis.org/oas/latest.html>

freeCodeCamp

Practice | Beginner | Free

Supplement notes with independent exercises; verify current course tracks.

<https://www.freecodecamp.org/learn/>

LeetCode study plans

DSA practice | Intermediate | Mixed free and paid

Practice patterns and explain complexity; avoid memorizing company tags as predictions.

<https://leetcode.com/studyplan/>

GreatFrontEnd

Interview practice | Intermediate to advanced | Mixed free and paid

Optional UI and frontend interview practice; repository exercises remain free.

<https://www.greatfrontend.com/>

Chrome DevTools

Browser debugging | Intermediate | Free

Capture traces and network evidence while debugging.

<https://developer.chrome.com/docs/devtools/>

Docker getting started

Deployment | Intermediate | Free docs; service plans vary

Run the sample MongoDB stack and learn volume persistence.

<https://docs.docker.com/get-started/>

Amazon how we hire

Company | All | Free

Confirm the role format and prepare evidence-based behavioral stories.

<https://www.amazon.jobs/content/en/how-we-hire>

Google hiring process

Company | All | Free

Review official process and ask the recruiter for your role format.

<https://www.google.com/about/careers/applications/how-we-hire/>

Microsoft technical interviewing

Company | All | Free

Use official preparation guidance; practice clarification and reasoning.

<https://careers.microsoft.com/v2/global/en/hiring-tips/technical-interviewing>